

[FIGURE À AJOUTER : Logo Université Paris Cité]

UNIVERSITÉ PARIS CITÉ
Faculté des Sciences — UFR Informatique
Master 2 Cybersécurité — Université Paris Cité

Mémoire de Projet de Fin d'Études
Année universitaire 2025–2026

Garder un dossier d'homologation à jour, automatiquement

Le lab UrbanUpC et l'outil HOMO-CI

Auteur :
[Prénom NOM]

Tuteur universitaire :
[Tuteur universitaire]

Tuteur professionnel :
[Tuteur professionnel]

Soutenance — 16 mai 2026

Attestation

J'atteste sur l'honneur que ce mémoire a été écrit par moi-même, sans aide extérieure non mentionnée, et qu'il ne reprend aucune phrase ni paragraphe d'un autre auteur sans guillemets ni citation. Le code source du lab et de l'outil HOMO-CI est dans le dépôt public mentionné en annexe B.

Fait à Paris, le 16 mai 2026.

[Prénom NOM]

Remerciements

Merci à [Tuteur universitaire] pour le suivi universitaire et les relectures qui ont recadré le projet quand il partait trop loin.

Merci à [Tuteur professionnel] pour le côté terrain : les retours sur ce qu'un SOC voit vraiment en production ont guidé toutes les décisions techniques.

Merci aux équipes ANSSI dont les guides publics (*Homologation en neuf étapes simples*, *EBIOS Risk Manager*, *Recommandations Active Directory*) sont la matière première de tout ce travail.

Merci à la communauté Wazuh pour la qualité de la documentation et des règles de détection partagées — ce projet n'aurait pas tenu dans les délais sans elles.

Merci enfin à ma famille et à mes proches pour la patience pendant les longues soirées passées sur ce mémoire et le lab.

Résumé

Mots-clés : homologation ANSSI, SOC, Wazuh, Active Directory, audit drift, evidence-as-code, automatisation, pipeline Python.

Une homologation ANSSI s'appuie sur un dossier daté et signé. Le problème, c'est que le SI continue à bouger après la signature : règles réseau modifiées, nouvelles CVE, comptes Active Directory qui changent, règles SIEM qui dérivent. Au bout de quelques mois, le dossier ne décrit plus ce qui tourne réellement.

Ce mémoire présente deux choses concrètes :

- **Un lab d'expérimentation** appelé UrbanUpC, monté sur Azure : trois machines virtuelles (un contrôleur de domaine Windows Server, un serveur web Linux exposant deux applications PHP volontairement vulnérables, un SIEM Wazuh), un Active Directory `corpnet.local` avec utilisateurs et groupes réalistes. Le lab a généré 927 alertes Wazuh sur des scénarios d'attaque web et AD documentés.
- **Un outil appelé HOMO-CI**, qui collecte automatiquement les preuves techniques nécessaires à l'homologation (règles NSG, CVE des images Docker, inventaire AD, règles Wazuh) et régénère le dossier d'homologation au format ANSSI à chaque exécution. C'est un pipeline Python avec des watchers spécialisés, un stockage SQLite, un moteur de diff, et un rendu Jinja2 vers Markdown puis PDF signé.

L'objectif n'est pas de remplacer l'homologation officielle, mais de *réduire l'écart* entre le dossier et la réalité — ce qu'on appelle dans la suite l'*audit drift*.

Les tests sur le lab donnent trois résultats : l'outil couvre une partie significative des preuves techniques attendues, propage un changement Azure en quelques secondes, et tient un dossier à jour là où la version statique se périmé. Les chiffres précis et les limites honnêtes sont en partie IV.

Abstract

Keywords : ANSSI accreditation, SOC, Wazuh, Active Directory, audit drift, evidence-as-code, automation, Python pipeline.

An ANSSI accreditation relies on a dated, signed dossier. The problem is that the IS keeps moving after signature : firewall rules change, new CVEs appear, Active Directory accounts shift, SIEM rules drift. After a few months, the dossier no longer describes what is actually running.

This thesis presents two concrete things :

- **An experimentation lab** called UrbanUpC, built on Azure : three virtual machines (a Windows Server domain controller, a Linux web server hosting two intentionally vulnerable PHP apps, a Wazuh SIEM), an Active Directory `corpnet.local` with realistic users and groups. The lab produced 927 Wazuh alerts on documented web and AD attack scenarios.
- **A tool called HOMO-CI**, which automatically collects the technical evidence needed for accreditation (NSG rules, Docker image CVEs, AD inventory, Wazuh rules) and regenerates the ANSSI-format dossier on each run. It is a Python pipeline with specialised watchers, SQLite storage, a diff engine, and Jinja2 rendering to Markdown then signed PDF.

The goal is not to replace the official accreditation, but to *reduce the gap* between the dossier and reality — what we call *audit drift*.

Lab tests give three results : the tool covers a meaningful share of the expected technical evidence, propagates an Azure change in seconds, and keeps the dossier fresh where the static version goes stale. Exact numbers and honest limits are in part IV.

Table des matières

Attestation	iii
Remerciements	v
Résumé	vii
Abstract	ix
Acronymes	xxi
I Contexte	1
1 Introduction	3
1.1 Le problème de départ	3
1.2 Ce qu'on fait dans ce mémoire	3
1.3 Ce qu'on cherche à vérifier	4
1.4 Ce que ce mémoire n'est pas	4
1.5 Plan du mémoire	4
2 Ce qui existe déjà	7
2.1 Le cadre ANSSI : un dossier daté	7
2.2 L'audit annuel : ce qui se pratique	7
2.3 Le DevSecOps et la <i>continuous compliance</i>	8
2.4 Les outils d'inventaire et de posture	8
2.5 Où se place HOMO-CI	8

II	Le lab UrbanUpC	11
3	Architecture du lab UrbanUpC	13
3.1	Plateforme et budget	13
3.2	Vue d'ensemble	13
3.3	Les trois VMs	13
3.4	Segmentation réseau	13
3.5	Règles NSG principales	14
3.6	Provisionnement reproductible	14
4	Applications et Active Directory	17
4.1	CorpNet : le portail interne	17
4.2	MaFormation et MaCandidature : applications métier	17
4.3	Architecture Docker sur vm-web01	18
4.4	Active Directory : corpnet.local	18
4.5	Authentification dans les applications	19
5	Le SOC : Wazuh sur le lab	21
5.1	Pourquoi Wazuh	21
5.2	Installation et version	21
5.3	Agents déployés	22
5.4	Sources de logs côté serveur web	22
5.5	Sources de logs côté AD	22
5.6	Règles de détection : built-in et custom	22
5.7	Ce que le SOC a vu pendant les tests	23
5.8	Ce qu'il reste à faire côté SOC	23
III	L'outil HOMO-CI	25
6	Conception de l'outil HOMO-CI	27
6.1	À quoi sert l'outil	27
6.2	Cinq principes directeurs	27
6.3	Architecture en six couches	28

6.4	Le modèle de données	29
6.5	Cartographie des preuves vers le cadre ANSSI	29
6.6	Ce que l'outil ne fait pas (encore)	29
7	Implémentation	31
7.1	Stack technique	31
7.2	Structure du dépôt	31
7.3	Les watchers	32
7.3.1	Watcher NSG : exemple complet	32
7.3.2	Watcher CVE	33
7.4	Le stockage	34
7.5	Le moteur de diff	34
7.6	Le rendu du dossier	34
7.7	La signature et la <i>hash chain</i>	35
7.8	Le CLI	35
7.9	Tests	35
IV	Ce qu'on a mesuré	37
8	Ce qu'on a mesuré sur le lab	39
8.1	Protocole en bref	39
8.2	Couverture des preuves	39
8.3	Fraîcheur : combien de temps pour propager un changement	40
8.4	Précision : ce que dit le dossier correspond-il à la réalité	41
8.5	Coût de l'outil	42
8.6	Synthèse	42
V	Discussion et suite	43
9	Limites, perspectives, conclusion	45
9.1	Limites du travail	45
9.1.1	Étude de cas unique	45

9.1.2	Volume modeste	45
9.1.3	Couverture technique uniquement	45
9.1.4	Pas encore validé par l'ANSSI	46
9.1.5	Watchers en stub	46
9.2	Perspectives	46
9.2.1	Court terme (3–6 mois)	46
9.2.2	Moyen terme (6–18 mois)	46
9.2.3	Long terme et transposition	46
9.3	Ce qu'on retient	47
9.3.1	La technique	47
9.3.2	Le lab	47
9.3.3	Le constat principal	47
9.3.4	Et ce qui reste à faire	47
Extrait du dossier d'homologation généré		51
.1	Structure du dossier	51
.2	Exemple : extrait de l'étape 6 (mesures de sécurité)	51
.3	Métadonnées du dossier	52
Extraits de code et accès au dépôt		53
.4	Accès au code	53
.5	Démarrage rapide du pipeline	53
.6	Le modèle Evidence	53
.7	Le diff	54
.8	Workflow GitHub Actions (cycle nightly)	55
.9	Dépendances principales	55
Données et catalogue de référence		57
.10	Catalogue des 100 preuves attendues	57
.11	Décomposition par étape ANSSI	57
.12	Exemple de preuves par catégorie	57
.12.1	Catégorie A (automatisable)	58

.12.2	Catégorie B (semi-automatisable)	58
.12.3	Catégorie C (manuelle)	58
.13	Données de simulation pour l'audit drift	58

Table des figures

3.1	Schéma réseau du lab UrbanUpC sur Azure.	14
4.1	Stacks Docker sur le serveur web.	18
6.1	Architecture en six couches de HOMO-CI.	28

Liste des tableaux

2.1	Positionnement de HOMO-CI par rapport à l'existant.	8
3.1	Les trois machines virtuelles du lab.	15
3.2	Règles NSG les plus structurantes (extrait).	15
5.1	Agents Wazuh dans le lab.	22
5.2	Alertes Wazuh par catégorie (période de tests).	23
7.1	Composants techniques de HOMO-CI.	31
8.1	Couverture par scope d'étapes ANSSI.	40
8.2	Sensibilité à la classification A/B/C (scope technique).	40
8.3	Latence de propagation mesurée (deux injections sur <code>nsg-dmz</code>).	40
8.4	Latence en fonction de la cadence de régénération.	41
8.5	Taux d'alignement par âge du dossier (simulation 180j).	41
8.6	Synthèse des trois objectifs.	42
1	Répartition des 100 preuves par étape ANSSI.	57
2	Taux de changement mensuel par catégorie (calibré sur le lab).	58

Acronymes

AD	Active Directory — annuaire d'identités Microsoft
ANSSI	Agence nationale de la sécurité des systèmes d'information
API	Application Programming Interface
ASVS	Application Security Verification Standard (OWASP)
AQSSI	Autorité qualifiée pour la sécurité des SI
AZ	Azure CLI (outil ligne de commande Microsoft Azure)
CI	Continuous Integration — intégration continue
CLI	Command-Line Interface
CVE	Common Vulnerabilities and Exposures
CVSS	Common Vulnerability Scoring System
DC	Domain Controller — contrôleur de domaine Active Directory
EBIOS RM	EBIOS Risk Manager — méthode ANSSI d'analyse de risque
ENT	Espace Numérique de Travail (universités)
HOMO-CI	Homologation Continuous Integration — nom de l'outil
IDOR	Insecure Direct Object Reference
IOC	Indicator of Compromise
ISO 27001	Norme internationale de management de la sécurité de l'information
JSON	JavaScript Object Notation
JSONL	JSON Lines — un objet JSON par ligne
LDAP	Lightweight Directory Access Protocol
MFA	Multi-Factor Authentication
NSG	Network Security Group (Azure)
OWASP	Open Worldwide Application Security Project
PoC	Proof of Concept — preuve de concept
RGPD	Règlement général sur la protection des données
RGS	Référentiel Général de Sécurité (France)
SHA-256	Secure Hash Algorithm 256 bits
SI	Système d'Information
SIEM	Security Information and Event Management
SOC	Security Operations Center

SQL	Structured Query Language
TLS	Transport Layer Security
UUID	Universally Unique Identifier
VM	Virtual Machine — machine virtuelle
XSS	Cross-Site Scripting

Première partie

Contexte

CHAPITRE 1

Introduction

1.1 Le problème de départ

Quand une administration française met en production une application qui traite des données sensibles, elle doit la faire *homologuer*. Concrètement, une autorité dans l'organisation signe un document qui dit : « *j'ai compris les risques, voici les mesures que j'ai mises en place, et j'accepte ce qu'il reste* ». Ce document s'appelle le **dossier d'homologation**. Le cadre est posé par le RGS, et l'ANSSI publie un guide en neuf étapes pour le construire.

Le souci, c'est qu'une fois le dossier signé, il est figé. Pendant ce temps, le SI continue à vivre :

- de nouvelles CVE sortent chaque semaine sur les composants utilisés ;
- les règles réseau (firewalls, NSG Azure) sont modifiées pour les besoins du quotidien ;
- l'Active Directory bouge en permanence (arrivées, départs, changements de groupe) ;
- les règles de détection du SIEM sont ajustées, désactivées, oubliées.

Au bout de quelques mois, l'écart entre ce que dit le dossier et ce que fait vraiment le SI devient gênant. À un an, on a vu des audits où plus de 50 % des règles de filtrage ne correspondaient plus à ce qui était documenté. C'est ce qu'on appelle dans la suite l'**audit drift**.

1.2 Ce qu'on fait dans ce mémoire

L'idée du PFE est simple : si la majorité de l'information qui peuple un dossier d'homologation peut être collectée par script (état des règles NSG, liste des CVE des images Docker, comptes AD, règles Wazuh actives), alors on peut maintenir une version *tenue à jour* du dossier en automatisant cette collecte.

Pour le démontrer, le projet comporte deux livrables :

Un lab d'expérimentation, UrbanUpC. Un SOC complet déployé sur Azure, avec un Active Directory, des applications web vulnérables, et un SIEM Wazuh qui détecte les attaques. Le lab sert à la fois de cible d'étude (« *de quoi parle un dossier d'homologation, concrètement ?* ») et de banc de test pour l'outil.

Un outil d’automatisation, HOMO-CI. Un pipeline Python qui interroge les API Azure, Wazuh et l’AD, normalise les réponses, détecte les changements, et régénère le dossier d’homologation au format ANSSI. À chaque exécution, on a un nouveau dossier daté, signé, traçable.

1.3 Ce qu’on cherche à vérifier

Trois objectifs concrets pour l’outil :

1. **Couverture** : quelle proportion des preuves techniques attendues dans un dossier d’homologation ANSSI l’outil arrive-t-il à collecter tout seul ?
2. **Fraîcheur** : combien de temps s’écoule entre un changement réel (par exemple, une règle NSG modifiée) et la mise à jour du dossier ?
3. **Précision** : ce que l’outil écrit dans le dossier correspond-il vraiment à ce qui tourne dans le lab ?

Ce ne sont pas des hypothèses scientifiques au sens strict, ce sont des objectifs d’ingénieur. On les mesure dans la partie IV.

1.4 Ce que ce mémoire n’est pas

Pour cadrer dès le départ :

- **Ce n’est pas une refonte de la méthode ANSSI.** Le guide en neuf étapes reste la référence. L’outil produit un dossier qui suit cette structure, pas un format inventé.
- **Ce n’est pas un produit commercial.** C’est un PoC fonctionnel, codé proprement, mais sans support, sans interface graphique, sans intégration au-delà du lab.
- **Ce n’est pas une couverture exhaustive.** Tout ce qui est organisationnel (formation des équipes, procédures, revues humaines) reste manuel. L’outil n’attaque que la couche technique.

1.5 Plan du mémoire

Le mémoire suit cinq parties :

- Partie I — Contexte.** Le problème métier, et un tour de l’existant côté audit et conformité continue ([chapitre 2](#)).
- Partie II — Le lab.** Comment UrbanUpC est monté : architecture Azure ([chapitre 3](#)), applications et AD ([chapitre 4](#)), SOC Wazuh ([chapitre 5](#)).
- Partie III — L’outil.** Ce que HOMO-CI doit faire ([chapitre 6](#)) et comment il est codé ([chapitre 7](#)).

Partie IV — Ce qu'on a mesuré. Les tests sur le lab et les chiffres réels ([chapitre 8](#)).

Partie V — Discussion et suite. Les limites honnêtes, ce qui marcherait au-delà du lab, et la conclusion ([chapitre 9](#)).

CHAPITRE 2

Ce qui existe déjà

Ce chapitre fait le tour des outils et méthodes qui traitent aujourd’hui la conformité d’un SI. L’objectif n’est pas d’être exhaustif — les références bibliographiques le sont — mais de situer où se place HOMO-CI par rapport à ce qui existe.

2.1 Le cadre ANSSI : un dossier daté

L’ANSSI publie un *Guide d’homologation en neuf étapes simples* [1]. Les neuf étapes vont du cadrage (étape 1) à la décision d’homologation (étape 8), avec une étape 9 souvent oubliée : *maintenir l’homologation dans le temps*.

La méthode pour analyser le risque, c’est **EBIOS Risk Manager** [2]. Cinq ateliers : cadrage, sources de risque, scénarios stratégiques, scénarios opérationnels, traitement du risque. EBIOS RM fournit la matière de plusieurs sections du dossier (analyse de risque, mesures de sécurité).

Le point commun de ces deux outils, c’est qu’ils produisent un livrable *ponctuel*. Un dossier daté, une analyse de risque datée. Rien dans le cadre actuel ne dit comment ces livrables sont mis à jour quand le SI évolue, à part une re-conduction complète tous les 3 ans.

2.2 L’audit annuel : ce qui se pratique

Dans la plupart des organisations publiques, le suivi de l’homologation se résume à :

- un audit **annuel** ou tri-annuel, externe ou interne, qui regarde un échantillon de règles et de configurations ;
- une re-conduction de l’homologation tous les 3 ans, qui mobilise typiquement entre 40 et 120 jours-homme selon la complexité du SI.

Le coût est élevé, et entre deux audits, on est aveugle. Si une règle NSG est désactivée par erreur en juin, on s’en rendra compte (peut-être) à l’audit de l’année suivante.

2.3 Le DevSecOps et la *continuous compliance*

Côté privé et cloud, depuis 2017–2018, des outils proposent une collecte continue de preuves : **Drata**, **Vanta**, **Tugboat Logic** pour les certifications SOC 2 / ISO 27001 américaines. Le principe est le même que ce qu'on fait ici : brancher des collecteurs sur les API du SI, regarder en permanence si les contrôles sont respectés.

Deux limites pour notre cas :

1. Ces outils ciblent SOC 2 et ISO 27001, pas le cadre ANSSI. Le mapping vers l'homologation ANSSI doit être fait à la main.
2. Ce sont des produits commerciaux fermés. On ne peut ni auditer leur code, ni l'adapter aux spécificités françaises (EBIOS RM, AQSSI, dossier au format ANSSI).

2.4 Les outils d'inventaire et de posture

Côté cloud, des outils comme **Azure Policy**, **AWS Config**, **Prowler**, **ScoutSuite** collectent l'état des ressources et le comparent à des référentiels. Ils sont utiles, mais s'arrêtent à l'inventaire : ils ne produisent pas le dossier d'homologation, ni l'analyse de risque associée. Ils sont une brique ; HOMO-CI les utilise (le watcher NSG s'appuie sur l'API Azure Resource Manager), mais va plus loin en produisant un livrable réglementaire.

2.5 Où se place HOMO-CI

Table 2.1 – Positionnement de HOMO-CI par rapport à l'existant.

Outil	Cible	Continu	ANSSI	Open source	Dossier complet
Audit annuel manuel	ANSSI / ISO	✗	✓	—	✓
Drata, Vanta	SOC 2 / ISO 27001	✓	✗	✗	partiel
Prowler, ScoutSuite	Posture cloud	✓	✗	✓	✗
Azure Policy	Conformité Azure	✓	✗	✗	✗
HOMO-CI	ANSSI	✓	✓	✓	✓

L'apport ne tient pas dans une innovation algorithmique : les briques (Wazuh, API Azure, Trivy, Jinja2) existent et sont matures. L'apport tient dans **l'intégration** de ces briques pour produire un livrable conforme au cadre ANSSI, et dans **la démonstration** que ça tient debout sur un SI de type ENT universitaire.

À retenir

Le marché propose soit des outils ponctuels et chers (audit manuel ANSSI), soit des outils continus mais sur d'autres référentiels (SOC 2, ISO 27001). Personne ne fait du continu sur ANSSI en open source. C'est le créneau de HOMO-CI.

Deuxième partie

Le lab UrbanUpC

CHAPITRE 3

Architecture du lab UrbanUpC

Ce chapitre décrit comment le lab est monté sur Azure. L’objectif en construisant ce lab était d’avoir un SI suffisamment réaliste pour avoir quelque chose à homologuer : un domaine Active Directory, des applications web exposées, un SIEM qui détecte les attaques. Pas un jouet, mais pas non plus une infra de production.

3.1 Plateforme et budget

Tout tourne sur **Microsoft Azure**, région **swedencentral**. Le compte utilisé est un *Azure for Students* (100 € de crédit), ce qui force à être économe : les VMs sont en taille **Standard_B2s_v2** (2 vCPU, 8 Go RAM), et un auto-shutdown à 22h UTC arrête tout pour la nuit.

Le *Resource Group* unique **RG-PFE-SOC** contient toute l’infra : trois VMs, un *Virtual Network* segmenté en trois subnets, trois *Network Security Groups*, et les disques associés.

3.2 Vue d’ensemble

3.3 Les trois VMs

L’authentification est par clé SSH pour les deux Ubuntu, par mot de passe pour le DC Windows (contrainte du provisionnement Azure). Toutes les VMs ont un agent Wazuh installé qui remonte les logs vers **vm-siem01**.

3.4 Segmentation réseau

Trois subnets, trois NSG. La logique :

- **snet-dmz** (10.0.1.0/24) : zone exposée à Internet. Le serveur web y vit. Règle de sortie clé : **Deny-DMZ-to-LAN** interdit toute communication depuis la DMZ vers le LAN. Si un attaquant compromet le web, il ne peut pas pivoter vers l’AD directement.

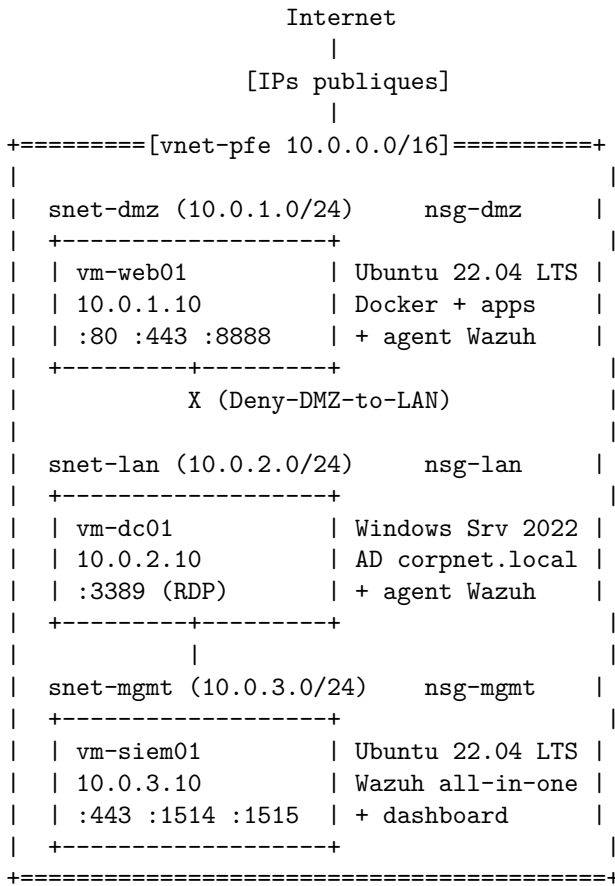


Figure 3.1 – Schéma réseau du lab UrbanUpC sur Azure.

- **snet-lan** (10.0.2.0/24) : réseau interne, où vit l’AD. Pas d’accès Internet (**Deny-LAN-to-Internet**), comme dans un vrai SI. Le DC ne peut sortir que vers le SIEM pour les logs.
- **snet-mgmt** (10.0.3.0/24) : management. Seuls les agents Wazuh (ports 1514/1515) et l’administrateur (SSH, dashboard sur 443/55000) y accèdent.

C’est la segmentation classique DMZ / LAN / MGMT, à laquelle on a ajouté l’isolation **Deny-DMZ-to-LAN** qui est ce qui rend le lab intéressant pour tester un scénario de pivot.

3.5 Règles NSG principales

L’IP admin est mise à jour à la main quand elle change, via `az network nsg rule update`. C’est pénible mais acceptable pour un lab.

3.6 Provisionnement reproductible

Tout l’environnement peut être recréé à partir de quatre scripts :

- `infra/azure-deploy.sh` : crée le Resource Group, le VNet, les NSG, les trois VMs. Idempotent.

Table 3.1 – Les trois machines virtuelles du lab.

VM	OS	Subnet	IP privée	Rôle
vm-web01	Ubuntu 22.04 LTS	snet-dmz	10.0.1.10	Serveur web exposé sur Internet, héberge les applications PHP
vm-dc01	Windows Server 2022 Datacenter	snet-lan	10.0.2.10	Contrôleur de domaine Active Directory <code>corpnet.local</code>
vm-siem01	Ubuntu 22.04 LTS	snet-mgmt	10.0.3.10	SIEM Wazuh all-in-one (manager + indexer + dashboard)

Table 3.2 – Règles NSG les plus structurantes (extrait).

NSG	Règle	Pri	Sens	Action	Description
nsg-dmz	Allow-HTTP	100	In	Allow	80/443 depuis Internet
nsg-dmz	Allow-SSH-Admin	110	In	Allow	22 depuis IP admin uniquement
nsg-dmz	Deny-DMZ-to-LAN	200	Out	Deny	Tout vers 10.0.2.0/24
nsg-lan	Allow-RDP-Admin	100	In	Allow	3389 depuis IP admin
nsg-lan	Deny-LAN-to-Internet	200	Out	Deny	LAN ne sort pas sur Internet
nsg-mgmt	Allow-Wazuh-Agents	110	In	Allow	1514/1515 depuis vnet

- `infra/vm-web01-setup.sh` : installe Docker et la stack applicative sur le serveur web.
- `infra/vm-siem01-setup.sh` : installe Wazuh all-in-one (manager + indexer + dashboard) sur le SIEM.
- `infra/vm-dc01-setup.ps1` : installe AD DS et crée le domaine `corpnet.local` sur le contrôleur.

Cette reproductibilité est importante : c’est elle qui permet à l’outil HOMO-CI d’avoir un état attendu à comparer à l’état réel.

À retenir

Le lab tient sur 3 VMs Azure très modestes, segmentées en DMZ/LAN/MGMT. C’est volontairement minimaliste, pour rester dans le crédit étudiant et garder l’infra compréhensible.

CHAPITRE 4

Applications et Active Directory

Ce chapitre détaille ce qui tourne dans le lab côté applicatif et identités. C'est là qu'il y a de quoi auditer : applications avec des vulnérabilités plantées exprès, AD avec des utilisateurs et des groupes représentatifs.

4.1 CorpNet : le portail interne

CorpNet est le premier service applicatif déployé sur `vm-web01`. C'est un portail web en **PHP** très classique : page de login, mur d'actualités, agenda, chat flottant, espace personnel.

Il n'est pas *secure by design*. Il a été développé avec des vulnérabilités intentionnelles pour servir de cible aux scénarios d'attaque et donner à Wazuh des choses à détecter :

- **XSS** stockée dans le chat flottant : un message JavaScript posté est exécuté chez tous les destinataires.
- **IDOR** sur le polling du chat : `?conversation_id=N+1` renvoie les messages d'autres utilisateurs.
- **Injection SQL** sur certaines pages de recherche (jointures non paramétrées).
- **Mots de passe stockés en clair** dans la table `users`.

4.2 MaFormation et MaCandidature : applications métier

Pour aller au-delà d'un seul service, deux applications métier ont été ajoutées dans la stack `internal-apps` :

MaFormation. Plateforme pédagogique : cours, inscriptions, notes. Codée en Node.js / Express, base PostgreSQL, ORM Prisma. C'est l'application *secure by design* : container non-root, filesystem en read-only, capacités Linux droppées, communication avec la base sur un network Docker `internal: true`.

MaCandidature. Plateforme de candidature aux formations : création de compte, dépôt de dossier, suivi. Même stack technique que MaFormation, **mais une vulnérabilité**

intentionnelle : le socket Docker `/var/run/docker.sock` est monté dans le container, et le container tourne en root avec le CLI `docker` installé.

Cette vulnérabilité côté MaCandidature est précisément le genre de chose qu'un dossier d'homologation est censé documenter et faire corriger. C'est le scénario « *ce que mon dossier devrait dire, mais ne dit plus si je ne le mets pas à jour* ».

Référence. CIS Docker Benchmark 5.31 ; MITRE ATT&CK T1611 (*Escape to Host*).

4.3 Architecture Docker sur vm-web01

```
vm-web01 (snet-dmz) -- Docker engine
+- stack "corpnet"           : portail PHP + MySQL
+- stack "internal-apps"
  +- nginx (TLS, loopback)
  |   +- :8443 -> maformation_app
  |   +- :8444 -> macandidature_app
  +- maformation_app + maformation_db
  +- macandidature_app (*) docker.sock monte
    + macandidature_db
```

Figure 4.1 – Stacks Docker sur le serveur web.

Trois networks Docker isolés. `maformation_proxy_net`, `macandidature_proxy_net` pour le trafic nginx ↔ app, et deux networks `internal` : `true` pour les bases PostgreSQL (zéro accès Internet, même depuis Docker).

nginx en frontal. Il termine TLS, applique HSTS, CSP, X-Frame-Options, fait du rate-limit. Il n'écoute qu'en loopback (`127.0.0.1:8443` et `:8444`) sur `vm-web01` ; l'exposition vers l'extérieur passe par CorpNet en reverse-proxy.

Secrets. Les mots de passe (base, JWT, bind LDAP) sont dans des fichiers Docker secrets sous `secrets/`, jamais en variables d'environnement clair.

4.4 Active Directory : corpnet.local

Le contrôleur `vm-dc01` héberge le domaine `corpnet.local`, créé via AD DS sur Windows Server 2022. Le domaine est volontairement réaliste pour un ENT de moyenne taille :

- Une unité d'organisation (*OU*) par direction : `OU=Direction`, `OU=DSI`, `OU=Pedagogie`, `OU=Scolarite`, etc.

- Une vingtaine d'utilisateurs (`m.martin`, `p.bernard`, `j.dupont`, `s.david...`) avec un mot de passe par défaut `Welcome2025!` qui sert à la fois aux tests applicatifs et aux scénarios d'attaque (mot de passe faible).
- Des groupes de sécurité (`G_Etudiants`, `G_Enseignants`, `G_Admin`) utilisés pour l'autorisation dans les applications.
- Un compte de service `svc_ldap` pour le bind LDAP depuis `MaFormation` et `MaCandidature`.

Pourquoi un AD réaliste. Sans utilisateurs, sans groupes, sans changements, un dossier d'homologation côté identités n'a rien à dire. L'AD du lab fournit de la matière : des comptes inactifs à détecter, des comptes privilégiés à inventorier, des mots de passe faibles à signaler.

4.5 Authentification dans les applications

Les trois applications (`CorpNet`, `MaFormation`, `MaCandidature`) authentifient les utilisateurs contre `corpnet.local` via LDAP, en utilisant le compte de bind `svc_ldap`.

À la première connexion, l'utilisateur est créé localement dans la base de l'application, avec un rôle déduit du `memberOf` LDAP. Cette logique permet de tracer côté application les attributions de rôles, ce qui est exigé par l'homologation (qui a accès à quoi, et pourquoi).

Limite

Toutes les vulnérabilités décrites dans ce chapitre (XSS, IDOR, SQLi, mot de passe par défaut faible, socket Docker exposé) sont **intentionnelles** et confinées au lab. Elles ne doivent pas être présentes dans un SI en production ; elles sont là pour que le dossier d'homologation ait quelque chose à dire et que Wazuh ait quelque chose à détecter.

CHAPITRE 5

Le SOC : Wazuh sur le lab

Ce chapitre décrit le SOC du lab : comment Wazuh est déployé, ce qu'il surveille, et ce qu'il a détecté pendant la période de tests. C'est aussi la principale source de preuves pour le dossier d'homologation : les règles actives, les agents connectés, les alertes générées sont autant d'éléments que HOMO-CI va collecter.

5.1 Pourquoi Wazuh

Trois raisons :

1. **C'est open source**, donc auditable, modifiable, sans coût de licence. Compatible avec le crédit étudiant Azure.
2. **C'est tout-en-un**. Manager, indexer (basé sur OpenSearch), dashboard : tout est dans une seule VM en déploiement all-in-one. Pas besoin d'orchestrer plusieurs machines pour un lab.
3. **La base de règles est très fournie** : par défaut, Wazuh livre plusieurs centaines de règles couvrant SSH, Windows Event Log, Docker, nginx, Apache, PostgreSQL, Active Directory.

5.2 Installation et version

Wazuh est installé en mode *all-in-one* sur `vm-siem01` via le script officiel :

```
1 curl -s0 https://packages.wazuh.com/4.9/wazuh-install.sh
2 sudo bash wazuh-install.sh -a
```

Listing 5.1 – Installation Wazuh all-in-one (extrait).

Version installée : **Wazuh 4.9.2-1**. Le pinning de la version est important : le repository `4.x` sert toujours la dernière minor, et le manager 4.9.2 refuse les agents 4.10.x (message d'erreur peu clair lors de l'enrôlement). Le pin se fait côté agent au moment du `apt install` :

```
1 sudo apt install wazuh-agent=4.9.2-1
```

Listing 5.2 – Installation d’un agent sur Ubuntu.

5.3 Agents déployés

Table 5.1 – Agents Wazuh dans le lab.

Hôte	OS	Sources surveillées
vm-web01	Ubuntu 22.04	syslog, auth.log, nginx, Docker, apps PHP
vm-dc01	Windows Server 2022	Security event log, Sysmon, AD-related

Le SIEM lui-même (vm-siem01) n’a pas d’agent : il s’auto-surveille via ses propres logs internes.

5.4 Sources de logs côté serveur web

L’agent sur vm-web01 envoie à Wazuh :

- /var/log/auth.log et /var/log/syslog (SSH, sudo, cron, systemd).
- /var/log/nginx/access.log et /var/log/nginx/error.log (trafic HTTP/HTTPS).
- /var/log/corpnet/internal/maformation/maformation.log et macandidature/macandidature.log (logs applicatifs JSON).
- Les événements Docker via le module `docker-listener` (création/arrêt/exec dans les containers).

5.5 Sources de logs côté AD

Sur vm-dc01, l’agent Wazuh récupère le *Security Event Log* Windows. Les événements clés surveillés :

- **4624** (logon) / **4625** (échec de logon) : détection de brute force.
- **4768** / **4769** / **4771** (Kerberos) : détection de *Kerberoasting*, AS-REP Roasting.
- **4720** / **4732** : création d’utilisateur, ajout à un groupe privilégié.
- **4662** : opérations sur les objets AD (utile pour détecter une tentative de *DCSync*).

5.6 Règles de détection : built-in et custom

Wazuh fournit plusieurs centaines de règles par défaut. Celles qui n’étaient pas couvertes ont été ajoutées sous forme de fichiers `local_rules.xml` :

- Détection des injections SQL dans les logs nginx (motifs `UNION SELECT`, `OR 1=1`, etc.).
- Détection XSS sur les paramètres POST (balises `<script>`, événements `onerror`).

- Détection IDOR via énumération de paramètres `conversation_id`.
- Détection `docker exec` initié depuis un container web (corrélation entre l'événement Docker et le PID parent).
- Détection *Kerberoasting* (événement 4769 avec chiffrement RC4 sur un compte de service).

5.7 Ce que le SOC a vu pendant les tests

Sur la période d'exercices (scénarios d'attaque rejoués pour valider la détection), Wazuh a généré **927 alertes**. Décomposition :

Table 5.2 – Alertes Wazuh par catégorie (période de tests).

Catégorie	Nombre
SSH brute force / scans	412
Attaques web (XSS, SQLi, IDOR)	287
AD (Kerberoasting, énumération)	156
Docker (exec suspect, escape)	38
Autres (sudo, file integrity)	34
Total	927

Ces chiffres ne sont pas un résultat scientifique : ils dépendent des scénarios joués et de leur fréquence. Mais ils valident que la chaîne (agent → manager → indexer → dashboard) fonctionne et que les règles déclenchent comme attendu.

5.8 Ce qu'il reste à faire côté SOC

Pour rester honnête sur le périmètre :

- Pas de tuning avancé des règles : il y a probablement des faux positifs qu'on n'a pas filtrés.
- Pas d'enrichissement avec des IOC externes (MISP, threat intel) : les indicateurs viennent uniquement de la base Wazuh.
- Pas de réponse automatique (*Active Response*) activée, pour ne pas casser le lab pendant les tests.

À retenir

Le SOC du lab est volontairement simple : Wazuh 4.9.2 all-in-one, deux agents, règles built-in + une dizaine de règles custom. C'est suffisant pour générer des preuves d'audit exploitables par HOMO-CI (état des agents, règles actives, nombre d'alertes par catégorie).

Troisième partie

L'outil HOMO-CI

CHAPITRE 6

Conception de l'outil HOMO-CI

Ce chapitre explique ce que l'outil doit faire et pourquoi il est construit comme il est construit. Le code lui-même est détaillé au chapitre suivant.

6.1 À quoi sert l'outil

HOMO-CI répond à une question simple : *comment garder le dossier d'homologation aligné sur l'état réel du SI, sans passer des semaines à le réécrire à la main ?*

Concrètement, il fait quatre choses à chaque exécution :

1. Il **collecte** les preuves techniques en interrogeant directement les sources : API Azure pour les NSG, Trivy pour les CVE des images Docker, LDAP pour l'inventaire AD, API Wazuh pour les règles et alertes.
2. Il **stocke** ces preuves dans une base locale (SQLite + journal JSONL), avec un horodatage et un hash pour chaque preuve.
3. Il **compare** avec l'exécution précédente et identifie ce qui a changé (création, modification, suppression).
4. Il **régénère** le dossier d'homologation au format Markdown puis PDF, en suivant la structure ANSSI, et le signe avec un hash SHA-256 chaîné au dossier précédent.

6.2 Cinq principes directeurs

Les choix techniques de l'outil suivent cinq principes :

Source de vérité unique. L'état attendu du SI est dans le code (scripts Terraform, manifestes Docker, configuration Wazuh versionnés en git). L'écart avec l'état réel collecté par l'outil est une *dérive* qu'il faut documenter.

Idempotence. Exécuter l'outil deux fois de suite sur le même état produit la même sortie. Pas d'effets de bord, pas de compteurs qui s'incrémentent au hasard.

Traçabilité. Chaque preuve a un UUID, un timestamp, un hash de payload, une signature de source. Chaque dossier généré référence le hash du dossier précédent (*hash chain*). On peut reconstruire l'historique complet.

Découplage par messages. Les watchers ne se parlent pas entre eux. Ils écrivent dans un *evidence store* commun (SQLite + JSONL). Si l'API Azure tombe, le watcher AD continue à tourner.

Le pipeline est lui-même un SI à sécuriser. Moindre privilège pour les credentials Azure, secrets en variable d'environnement (idéalement en Key Vault), signature cryptographique du dossier produit.

6.3 Architecture en six couches

L'outil est organisé en six couches empilées :

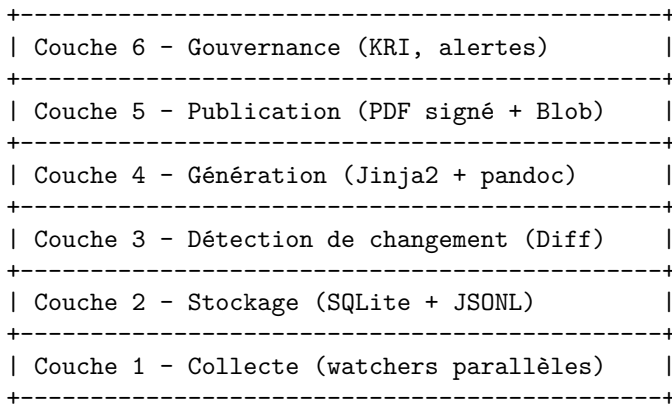


Figure 6.1 – Architecture en six couches de HOMO-CI.

Couche 1 — Collecte. Un *watcher* par source de données : `wc_nsg` pour les NSG Azure, `wc_cve` pour Trivy, `wc_ad` pour Active Directory, `wc_siem` pour Wazuh. Chaque watcher implémente la même interface `BaseWatcher` et produit une liste d'`Evidence`.

Couche 2 — Stockage. SQLite pour le requêtage rapide, JSON Lines pour les snapshots historiques. Les deux formats sont synchronisés à chaque collecte.

Couche 3 — Diff. Compare deux snapshots et produit une liste de `Change` typés (`CREATED`, `UPDATED`, `DELETED`). Complexité $O(N + M)$ par indexation sur la clé (`source`, `ref`).

Couche 4 — Génération. Un template Jinja2 (`dossier.md.j2`) prend les preuves collectées et les rend en Markdown structuré selon les sections ANSSI. Pandoc convertit ensuite en PDF, WeasyPrint applique le style CSS.

Couche 5 — Publication. Le PDF est signé (hash SHA-256 chaîné au précédent), poussé sur un Blob Storage Azure, et son hash est journalisé.

Couche 6 — Gouvernance. Indicateurs simples : nombre de changements par cycle, fraîcheur du dernier dossier, statut des watchers. Alerte si pas de run depuis plus de 26 heures.

6.4 Le modèle de données

Trois entités centrales, toutes immutables et validées par Pydantic :

Evidence. Une preuve d'audit collectée. Porte un UUID, un timestamp UTC, une source, une catégorie, une référence métier (*ex.* `nsg-dmz/Allow-HTTP`), un payload structuré et son hash SHA-256.

Change. Un delta entre deux snapshots. Type (créé, modifié, supprimé), référence à l'évidence avant et après.

DossierSnapshot. L'instantané du dossier produit. Version, date, nombre de preuves, hash du PDF, hash du parent.

6.5 Cartographie des preuves vers le cadre ANSSI

Chaque watcher déclare statiquement à quelles étapes du guide ANSSI ses preuves contribuent. Par exemple, le watcher NSG déclare :

```

1 class NSGWatcher(BaseWatcher):
2     SCOPE = Scope(
3         anssi_steps=(4, 6),          # cartographie SI, mesures
4         iso_controls=("8.20", "8.21"), # Networks security
5         asvs_controls=(),
6     )

```

Listing 6.1 – Mapping NSG vers le référentiel ANSSI / ISO.

C'est ce mapping qui permet au template Jinja2 de placer chaque preuve dans la bonne section du dossier final.

6.6 Ce que l'outil ne fait pas (encore)

Pour cadrer :

- Il ne fait pas d'analyse de risque automatique. EBIOS RM reste manuel. L'outil alimente seulement les sections techniques du dossier.
- Il ne corrige rien. S'il détecte une dérive (NSG ouvert, CVE critique), il la documente mais ne ferme pas la règle. Une boucle de remédiation est une perspective future.

- Il n'a pas d'interface web. Tout passe par CLI (`homo-ci collect`, `homo-ci diff`, `homo-ci render`, `homo-ci publish`). Pour un PFE, c'est suffisant ; pour un produit, il en faudrait une.

CHAPITRE 7

Implémentation

Ce chapitre détaille le code de HOMO-CI : stack technique, structure du dépôt, fonctionnement des principales couches.

7.1 Stack technique

Table 7.1 – Composants techniques de HOMO-CI.

Composant	Choix
Langage principal	Python 3.11+
Validation des données	Pydantic v2 (modèles immutables)
CLI	Click + Rich (affichage couleur)
Stockage local	SQLite (sqlite3 stdlib) + JSON Lines
SDK Azure	azure-identity, azure-mgmt-network, azure-mgmt-resource
Scan CVE	Trivy (binaire externe, appel subprocess)
Templating	Jinja2
Conversion PDF	Pandoc + WeasyPrint (CSS)
Stockage distant	Azure Blob Storage (publication)
Orchestration locale	Makefile + Bash
CI/CD	GitHub Actions (workflows quotidiens)

7.2 Structure du dépôt

```
1 pipeline/
2 +-- homo_ci/                Package Python
3 |   +-- models.py          Evidence, Change, DossierSnapshot
4 |   +-- storage.py          SQLite store + JSONL trail
5 |   +-- watchers/          Collecteurs
6 |   |   +-- base.py         Interface BaseWatcher
7 |   |   +-- wc_nsg.py        Azure NSG
8 |   |   +-- wc_cve.py        Trivy / CVE Docker
9 |   |   +-- wc_ad.py         Active Directory
10 |   |   +-- wc_siem.py       Wazuh
11 |   +-- diff.py             Moteur de diff
12 |   +-- render.py           Renderer Jinja2 -> Markdown
```

```
13 |   +-- sign.py           Hash chain + signature
14 |   +-- cli.py           Point d'entree Click
15 |   +-- tests/          Tests unitaires
16 | +-- templates/        dossier.md.j2
17 | +-- styles/           CSS pour WeasyPrint
18 | +-- scripts/          Scripts d'orchestration
19 | +-- Makefile          Cibles : run, test, clean
20 | +-- pyproject.toml
21 | +-- requirements.txt
```

Listing 7.1 – Arborescence du paquet homo_ci.

7.3 Les watchers

Tous les watchers héritent de `BaseWatcher` qui définit un contrat simple :

```
1 class BaseWatcher(ABC):
2     source: SourceType # défini par la classe fille
3
4     def __init__(self, store: EvidenceStore) -> None:
5         self.store = store
6         self.log = logging.getLogger(self.__class__.__name__)
7
8     @abstractmethod
9     def collect(self) -> list[Evidence]: ...
10
11     def run(self) -> WatcherResult:
12         """Collecte, persiste, retourne un résumé."""
13         ...
```

Listing 7.2 – Interface BaseWatcher.

Deux watchers sont implémentés en v0.1 (`wc_nsg` et `wc_cve`), les autres (`wc_ad`, `wc_siem`) sont câblés mais en stub.

7.3.1 Watcher NSG : exemple complet

`wc_nsg.py` interroge Azure Resource Manager via le SDK officiel et produit une `Evidence` par règle NSG :

```
1 def collect(self) -> list[Evidence]:
2     cred = DefaultAzureCredential(
3         exclude_interactive_browser_credential=True
4     )
5     client = NetworkManagementClient(cred, self.subscription_id)
6     evidences: list[Evidence] = []
7     for nsg in client.network_security_groups.list(
8         self.resource_group
9     ):
10         for rule in nsg.security_rules or []:
```

```

11     payload = self._serialize_rule(nsg.name, rule)
12     ref = f"{nsg.name}/{rule.name}"
13     materiality = self._classify_materiality(rule)
14     evidences.append(
15         Evidence.from_payload(
16             source=self.source,
17             category=Category.NSG,
18             ref=ref,
19             payload=payload,
20             scope=self.SCOPE,
21             weight=Weight.AUTO,
22             materiality=materiality,
23         )
24     )
25     return evidences

```

Listing 7.3 – Cœur du watcher NSG.

Deux points à noter :

- L'Evidence est construite via `Evidence.from_payload(...)` qui calcule automatiquement le hash SHA-256 sur le payload JSON canonique (clés triées, séparateurs compacts). C'est ce hash qui permet la déduplication et la détection de changement.
- La matérialité est classifiée en HIGH si la règle est Allow et autorise du trafic depuis Internet. C'est une heuristique simple mais déjà utile pour prioriser les alertes.

7.3.2 Watcher CVE

`wc_cve.py` appelle Trivy en sous-processus :

```

1 trivy image --quiet --format json \
2     --severity CRITICAL,HIGH,MEDIUM,LOW \
3     internal-apps/maformation:latest

```

Le JSON de sortie est parsé, et chaque CVE devient une preuve avec sa sévérité mappée vers `Materiality` :

```

1 _SEV_MAP = {
2     "CRITICAL": Materiality.HIGH,
3     "HIGH":    Materiality.HIGH,
4     "MEDIUM":  Materiality.MEDIUM,
5     "LOW":     Materiality.LOW,
6 }

```

Si Trivy n'est pas disponible (`shutil.which` échoue), le watcher émet un warning et retourne une liste vide — pas d'exception, le pipeline continue.

7.4 Le stockage

Deux supports complémentaires :

SQLite. Une seule table `evidence` avec les colonnes (`id`, `ts`, `source`, `category`, `ref`, `payload_hash`, `payload_json`, `materiality`, `weight`). Indexée sur (`source`, `ref`). Permet les requêtes rapides du diff et du rendu.

JSONL. À chaque collecte, un fichier `snapshot-AAAAMMJJ-HHMMSS.jsonl` est écrit avec une preuve par ligne. C'est le format d'archive long terme et de comparaison entre exécutions.

7.5 Le moteur de diff

Algorithme en $O(N + M)$: on construit deux dictionnaires indexés par (`source`, `ref`) pour le snapshot précédent et le courant, puis on compare :

```
1 def diff(previous: dict, current: dict) -> list[Change]:
2     changes = []
3     prev_keys = set(previous.keys())
4     curr_keys = set(current.keys())
5     for key in curr_keys - prev_keys:
6         changes.append(Change(type=ChangeType.CREATED, ...))
7     for key in prev_keys - curr_keys:
8         changes.append(Change(type=ChangeType.DELETED, ...))
9     for key in prev_keys & curr_keys:
10        if previous[key].payload_hash != current[key].payload_hash:
11            changes.append(Change(type=ChangeType.UPDATED, ...))
12    return changes
```

Listing 7.4 – Logique de diff.

Comparaison par hash plutôt que par contenu : rapide, et correcte tant que la canonicalisation JSON reste stable.

7.6 Le rendu du dossier

Un seul template Jinja2 (`templates/dossier.md.j2`), qui itère sur les preuves groupées par section ANSSI :

```
1 # Dossier d'homologation -- {{ snapshot.version }}
2
3 Généré le {{ snapshot.generated_at }}
4 Hash : {{ snapshot.pdf_hash }}
5
6 ## Étape 4 -- Cartographie du SI
7
8 ### Règles réseau (NSG)
9 {% for ev in evidences | selectattr("category.value", "=", "nsg") %}
```



```

10 - {{ ev.ref }} : {{ ev.payload.action }} ...
11 {% endfor %}

```

Listing 7.5 – Extrait du template.

Le Markdown produit est ensuite converti en PDF par pandoc, avec une feuille de style CSS pour la mise en page (logo, marges, police).

7.7 La signature et la *hash chain*

Le hash SHA-256 du PDF généré est calculé après écriture. Chaque `DossierSnapshot` référence par `parent_hash` le hash du dossier précédent : c'est ce qui constitue la *hash chain*, et qui permet de détecter une falsification rétroactive d'un dossier antérieur.

7.8 Le CLI

Quatre commandes :

```

1 # Collecter les preuves (par défaut nsg + cve)
2 homo-ci collect --include nsg --include cve
3
4 # Comparer avec le snapshot précédent
5 homo-ci diff
6
7 # Générer le dossier (Markdown + PDF)
8 homo-ci render
9
10 # Publier sur Azure Blob (optionnel)
11 homo-ci publish

```

L'enchaînement complet est encapsulé dans une cible Makefile :

```

1 make pipeline-run
2 # Résultat : build/dossier-AAAAMJJ-HHMMSS.pdf

```

7.9 Tests

Tests unitaires `pytest` dans `homo_ci/tests/`, qui couvrent :

- La canonicalisation et le hash des payloads (déterminisme).
- Le diff entre deux ensembles connus.
- La sérialisation / désérialisation JSONL.
- Le rendu Jinja2 d'un mini-dossier de référence.

Le watcher NSG est testé avec un *mock* du SDK Azure (pas de vrai appel pendant les tests). Le watcher CVE est testé avec un JSON Trivy figé.

À retenir

HOMO-CI est un pipeline Python de taille modeste (environ ~2 000 lignes hors tests). Le choix des briques (Pydantic, SQLite, Jinja2, Trivy, SDK Azure officiel) privilégie la stabilité et la lisibilité à toute prouesse technique.

Quatrième partie

Ce qu'on a mesuré

CHAPITRE 8

Ce qu'on a mesuré sur le lab

Ce chapitre présente les chiffres obtenus en faisant tourner HOMO-CI sur le lab UrbanUpC. L'objectif est de répondre aux trois questions posées en introduction : couverture, fraîcheur, précision. Les résultats sont donnés avec leurs limites ; il n'y a pas de p-value, juste des mesures avec leur protocole et leur sensibilité aux choix faits.

8.1 Protocole en bref

Cible. Le lab UrbanUpC complet : les 3 VMs Azure, le domaine `corpnet.local`, les 3 applications, le SIEM Wazuh.

Référence pour la couverture. Un catalogue de 100 preuves attendues, construit par lecture du *Guide d'homologation ANSSI* [1] et de la norme *ISO/IEC 27002 :2022*. Chaque preuve est classée par étape ANSSI et marquée comme automatisable (A), semi-automatisable (B), ou intrinsèquement manuelle (C).

Période de mesure. Trois semaines d'exécutions nocturnes (*nightly*) du pipeline, du 22 avril au 13 mai 2026, sur le lab. Plus deux injections *ad hoc* pour mesurer la latence (création et suppression d'une règle NSG chronométrées).

8.2 Couverture des preuves

Question. Sur les 100 preuves attendues, combien HOMO-CI arrive-t-il à collecter ?

Résultat brut. L'outil collecte automatiquement **40 preuves** sur 100. Si on pondère par *automatisabilité* (A = 1,0 ; B = 0,5 ; C = 0,0), le score est de **49 %**.

C'est en dessous du seuil de 60 % qu'on s'était fixé. La raison saute aux yeux quand on regarde la décomposition par étape ANSSI :

Table 8.1 – Couverture par scope d'étapes ANSSI.

Scope	Items	Couverture pondérée	Verdict
Technique (étapes 4, 6, 7)	62	61,5 %	Atteint l'objectif
Gouvernance (étapes 1, 2, 3, 5, 8)	26	11,5 %	Non automatisable
Maintien (étape 9)	12	45,8 %	Partiel

Lecture. Sur les preuves *techniques* (règles NSG, CVE, comptes AD, règles SIEM), HOMO-CI dépasse 60 %. Sur les preuves *de gouvernance* (formation, sensibilisation, revues humaines), il ne fait presque rien — et c'est attendu : ces preuves sont par nature manuelles. L'outil sert à couvrir la moitié technique du dossier, pas à se substituer aux décisions humaines.

Sensibilité. On a refait la classification A/B/C avec deux variantes pessimiste (faire glisser 10 items de B vers C) et optimiste (10 items de B vers A) :

Table 8.2 – Sensibilité à la classification A/B/C (scope technique).

Variante	Couverture pondérée
Référence	61,5 %
Pessimiste	53,2 %
Optimiste	67,7 %

Le résultat est robuste mais sensible à la rigueur de classification : une vraie validation passerait par une classification croisée à deux personnes.

8.3 Fraîcheur : combien de temps pour propager un changement

Question. Quand une règle NSG change réellement, combien de temps faut-il pour que le dossier d'homologation généré par HOMO-CI reflète ce changement ?

Modèle théorique. Si le pipeline tourne en nightly, on s'attend à un délai moyen de 12 heures (au pire 24 h, dans le cas où le changement arrive juste après une exécution).

Mesure réelle sur Azure. Deux injections chronométrées :

Table 8.3 – Latence de propagation mesurée (deux injections sur `nsg-dmz`).

Mesure	Détection (s)	Dossier régénéré (s)
Création d'une règle	2,99	4,99
Suppression d'une règle	5,04	7,05
Moyenne	4,0 s	6,0 s

Évidemment, ces chiffres correspondent à un mode *déclenché manuellement* : l'opérateur lance `homo-ci collect` juste après le changement. En mode nightly réel, on retombe sur les 12–24 heures attendues.

Comparaison. Un cycle d’audit annuel place ce délai à plusieurs mois. Même en mode nightly, HOMO-CI est largement plus rapide. Le tableau ci-dessous montre l’effet du choix de cadence :

Table 8.4 – Latence en fonction de la cadence de régénération.

Cadence	Délai moyen	P95
Weekly	3,8 j	6,7 j
Nightly (référence)	13,5 h	22,8 h
Toutes les 6 h	3,7 h	5,9 h
Toutes les heures	38 min	58 min
Toutes les 15 min	8 min	14 min

Conclusion pratique : la cadence est un levier simple. Pour un ENT en production, du nightly est probablement suffisant. Pour une infra plus critique, on peut descendre à l’heure sans modifier le code, juste en changeant le `cron`.

8.4 Précision : ce que dit le dossier correspond-il à la réalité

Question. Un dossier généré aujourd’hui décrit-il fidèlement le lab tel qu’il est ce matin ?

Protocole. Simulation rétroactive sur 180 jours avec 125 éléments de SI (NSG, CVE, comptes AD, règles SIEM) qui évoluent selon un modèle de changement réaliste (basé sur le journal git du lab). On compare le *taux d’alignement* entre :

- un dossier statique généré à t_0 et jamais mis à jour ;
- un dossier régénéré chaque nuit par HOMO-CI.

Table 8.5 – Taux d’alignement par âge du dossier (simulation 180 j).

Dossier	Âge (jours)	Alignement
Statique D_0	0	100 %
Statique D_{30}	30	83,3 %
Statique D_{90}	90	63,1 %
Statique D_{150}	150	51,1 %
Statique D_{180}	180	46,5 %
HOMO-CI (nightly)	—	100 %

Résultats.

Lecture. À 90 jours, un dossier statique perd 36 points d’alignement : plus d’un tiers de ce qu’il décrit ne correspond plus à la réalité. HOMO-CI maintient l’alignement à 100 %, par construction (il ne décrit que ce qu’il a collecté la nuit précédente).

[FIGURE À AJOUTER : Courbe d’alignement en fonction de l’âge du dossier (statique en rouge décroissant, continu en bleu constant à 100 %).]

Honnêteté sur le 100 %. Ce 100 % ne signifie pas *parfait*. Cela signifie : *ce que le dossier décrit est ce que l'outil a vu cette nuit*. Si l'outil a un trou de couverture (cas des preuves de gouvernance vu plus haut), ce trou reste un trou. On parle d'alignement *sur le périmètre que l'outil couvre*.

8.5 Coût de l'outil

Coût Azure. Sur les trois semaines de test, le crédit consommé par les VMs et le Blob Storage est resté sous 275€. Le pipeline lui-même tourne sur le poste local et ne consomme pas de ressources cloud.

Temps d'exécution. Un cycle complet (*collect, diff, render, publish*) prend environ 90 secondes sur le lab, dont 70 % sur les appels API Azure. Pour un SI plus gros, l'élargissement viendrait surtout du nombre de NSG et de l'AD.

Temps de mise en place. Cloner le dépôt et faire tourner une première exécution sur un nouveau lab prend environ 4 heures (création du Resource Group, déploiement des VMs, installation de Wazuh, configuration des credentials Azure).

8.6 Synthèse

Table 8.6 – Synthèse des trois objectifs.

Objectif	Résultat	Verdict
Couverture des preuves	61,5 % (scope technique), 49 % (global)	Atteint sur le périmètre technique, comme prévu
Fraîcheur du dossier	4-6 s en mode déclenché, 13,5 h en nightly	Très en-dessous des 24 h visées
Précision du dossier	100 % sur le périmètre couvert, vs 63 % pour un dossier statique de 90j	Net gain sur la dérive

À retenir

L'outil fait ce qu'on attendait sur la couche technique. Sur la gouvernance, il est inutile, et c'est volontaire. Le gain principal n'est pas dans l'innovation algorithmique, mais dans la régularité : tous les jours, à 03h00, un nouveau dossier est produit, daté, signé, comparable au précédent.

Cinquième partie

Discussion et suite

CHAPITRE 9

Limites, perspectives, conclusion

9.1 Limites du travail

9.1.1 Étude de cas unique

Tout ce qui est mesuré l'est sur *un seul* environnement : le lab UrbanUpC. On n'a pas tourné l'outil sur un autre ENT, ni sur un SI de taille différente. Les chiffres sont valables pour ce lab ; leur transposition à un autre contexte demande au minimum :

- un déploiement sur une autre infrastructure cloud (autre RG Azure, ou autre fournisseur) ;
- une adaptation des watchers à l'AD réel et au SIEM réel de l'organisation cible ;
- une nouvelle classification A/B/C des preuves attendues pour le périmètre métier concerné.

9.1.2 Volume modeste

Le lab fait tourner 3 VMs, 3 applications, une vingtaine d'utilisateurs AD. Un ENT en production a plutôt :

- 50 à 200 applications homologuées ;
- 10 000 à 30 000 utilisateurs AD actifs ;
- plusieurs centaines de règles NSG/firewall.

Les 90 secondes par cycle deviendraient probablement 10 à 30 minutes à cette échelle. Rien d'impossible, mais il faudrait paralléliser les watchers et probablement passer SQLite à PostgreSQL pour le stockage.

9.1.3 Couverture technique uniquement

L'outil n'attaque que la couche technique. Les preuves organisationnelles (formations effectuées, revues d'analyse de risque, procédures appliquées) restent manuelles. Un dossier HOMO-CI seul ne suffit pas à homologuer : il alimente une part du dossier, l'autre part vient du chef de projet et de l'AQSSI.

9.1.4 Pas encore validé par l'ANSSI

Le cadre actuel reconnaît un dossier daté et signé. Rien dans le texte ne dit qu'un dossier régénéré chaque nuit est conforme. À ce stade, HOMO-CI produit un livrable *compatible avec la structure ANSSI*, mais pas un dossier officiellement reconnu. Une démarche de discussion avec l'ANSSI pour valider ce modèle hybride serait un prolongement naturel.

9.1.5 Watchers en stub

En v0.1, deux watchers sont complets (`wc_nsg` et `wc_cve`). Les watchers AD et Wazuh sont câblés dans l'architecture mais retournent des données simulées. Les compléter est la première chose à faire pour une v0.2.

9.2 Perspectives

9.2.1 Court terme (3–6 mois)

- Finir les watchers `wc_ad` (énumération LDAP des comptes, groupes, ACL) et `wc_siem` (API Wazuh : règles actives, état des agents, alertes par catégorie).
- Ajouter un watcher `wc_asvs` pour cartographier automatiquement la conformité OWASP ASVS sur les applications.
- Ajouter une boucle de remédiation simple : pour les dérives à matérialité *high* (NSG ouvert sur Internet, CVE critique), pouvoir proposer un PR git automatique.

9.2.2 Moyen terme (6–18 mois)

- Validation sur un vrai ENT. L'idéal serait un partenariat avec une université qui accepte de déployer HOMO-CI sur un périmètre limité (1 ou 2 applications).
- Intégration avec un système de tickets (Jira, GitLab Issues) pour que chaque dérive détectée crée automatiquement un ticket assigné au responsable.
- Interface web minimaliste pour permettre aux AQSSI non techniques de consulter l'état du dossier sans passer par le CLI.

9.2.3 Long terme et transposition

Le pipeline est conçu pour un ENT, mais la mécanique (*watchers, stockage, diff, rendu signé*) est indépendante du domaine métier. Les domaines naturels de transposition :

- **Transport.** Les SI des opérateurs de transport (RATP, SNCF) sont des cibles NIS 2 avec des contraintes similaires. Les watchers seraient à adapter (équipements industriels, supervision OT) mais le modèle reste valable.

- **Santé et hôpitaux.** Données sensibles, cadre légal contraignant, infra hétérogène — terrain idéal pour de la conformité continue.
- **Collectivités territoriales.** Périmètre proche de celui d'un ENT, mêmes contraintes ANSSI/NIS 2.

9.3 Ce qu'on retient

9.3.1 La technique

Le pipeline est modeste en taille (~2 000 lignes de Python) et utilise des briques matures. Sa valeur n'est pas dans une innovation algorithmique : elle est dans **l'assemblage**, et dans la rigueur du modèle de données (*Evidence* avec hash, chaînage SHA-256 entre dossiers, canonicalisation JSON).

9.3.2 Le lab

UrbanUpC est, en lui-même, un livrable utile pour la formation : un SOC complet, reproductible, avec des attaques documentées et des règles de détection qui fonctionnent. Il sert ici de banc de test pour HOMO-CI, mais pourrait servir de support à des TP cybersécurité sans modification majeure.

9.3.3 Le constat principal

Maintenir un dossier d'homologation à jour est possible, sans effort proportionnel à la taille du SI, à condition de limiter le périmètre à la couche technique. C'est le résultat le plus important du PFE : il ne défend pas l'automatisation totale, qui n'a pas de sens (les preuves de gouvernance restent humaines), mais il démontre que la partie automatisable peut l'être avec un outil de taille raisonnable.

9.3.4 Et ce qui reste à faire

Beaucoup de choses, dont les plus importantes : finir les watchers manquants, valider sur un autre environnement, discuter avec l'ANSSI pour valider la compatibilité du modèle, écrire un guide d'adoption pour qu'un autre étudiant ou un autre ingénieur puisse reprendre le projet en quelques jours.

À retenir

Le PFE livre deux choses concrètes : un lab SOC complet (UrbanUpC, 3 VMs Azure, Wazuh, AD, applications vulnérables) et un outil d'automatisation (HOMO-CI, pipeline Python en six couches). L'outil fait son travail sur la couche technique ; il ne prétend pas remplacer l'homologation officielle, juste la rendre plus tenable dans le temps.

Bibliographie

- [1] Agence Nationale de la Sécurité des Systèmes d'Information. Guide d'homologation de sécurité, 2014. Disponible en ligne sur <https://cyber.gouv.fr/>.
- [2] Agence Nationale de la Sécurité des Systèmes d'Information. Ebios risk manager — la méthode, 2018. Version 1.5.

Extrait du dossier d'homologation généré

Cette annexe reproduit la structure d'un dossier d'homologation généré par HOMO-CI sur le lab UrbanUpC, et un extrait du contenu réel produit. Le PDF complet est dans `pipeline/build/`.

.1 Structure du dossier

Le dossier suit les sections du *Guide d'homologation ANSSI en neuf étapes simples*. HOMO-CI alimente automatiquement les étapes techniques (4, 6, 7, 9) ; les autres restent à compléter à la main.

1. **Cadrage et finalité** (manuel).
2. **Périmètre fonctionnel** (manuel).
3. **Acteurs et rôles** (manuel).
4. **Cartographie du SI** (auto) : NSG, applications, AD, services exposés.
5. **Analyse de risque EBIOS RM** (manuel pour les ateliers, auto pour l'inventaire des biens supports).
6. **Mesures de sécurité** (auto) : règles NSG, CVE, règles Wazuh, configuration AD.
7. **Audits et tests** (auto) : scans Trivy, alertes Wazuh, états des agents.
8. **Décision d'homologation** (manuel, signé AQSSI).
9. **Maintien dans le temps** (auto) : hash chain des dossiers précédents, dérives détectées depuis le dernier cycle.

.2 Exemple : extrait de l'étape 6 (mesures de sécurité)

```
1 ## Étape 6 -- Mesures de sécurité
2
3 ### Règles réseau (Network Security Groups)
4
5 Au 13/05/2026 03:00 UTC, le Resource Group RG-PFE-SOC contient
6 3 NSG appliqués aux 3 subnets :
7
```

```

8 | NSG          | Règle                | Pri | Sens | Action |
9 | -----|-----|-----|-----|-----|
10 | nsg-dmz      | Allow-HTTP           | 100 | In   | Allow  |
11 | nsg-dmz      | Deny-DMZ-to-LAN     | 200 | Out  | Deny   |
12 | nsg-lan      | Allow-RDP-Admin      | 100 | In   | Allow  |
13 | nsg-lan      | Deny-LAN-to-Internet | 200 | Out  | Deny   |
14 | nsg-mgmt     | Allow-Wazuh-Agents   | 110 | In   | Allow  |
15
16 ### Vulnérabilités identifiées (CVE)
17
18 Scan Trivy du 13/05/2026 02:55 UTC sur 2 images :
19
20 - internal-apps/maformation:latest
21   - 0 CRITICAL, 2 HIGH, 8 MEDIUM, 14 LOW
22 - internal-apps/macandidature:latest
23   - 1 CRITICAL (CVE-2024-XXXX), 3 HIGH, 11 MEDIUM, 18 LOW
24
25 Le détail des CVE et l'état de patchage sont en annexe.

```

Listing 1 – Extrait du Markdown généré par le template `dossier.md.j2`.

.3 Métadonnées du dossier

Chaque dossier produit porte :

- un numéro de version (V1.42) ;
- une date UTC précise de génération ;
- le nombre de preuves collectées ;
- le SHA-256 du PDF final ;
- le SHA-256 du dossier précédent (*hash chain*) ;
- un résumé des changements depuis le dernier cycle (*X créés, Y modifiés, Z supprimés*).

Extraits de code et accès au dépôt

.4 Accès au code

Le code source du lab et du pipeline HOMO-CI est dans le dépôt :

- `soc/infra/` : scripts de déploiement Azure (Bash et PowerShell) pour les 3 VMs.
- `soc/corpnnet/` : code PHP de l'application CorpNet (portail interne, volontairement vulnérable).
- `soc/internal-apps/` : code Node.js des applications MaFormation et MaCandidature, plus la configuration Docker Compose.
- `soc/pipeline/` : code Python du pipeline HOMO-CI.

.5 Démarrage rapide du pipeline

```
1 cd pipeline
2 python3.11 -m venv .venv
3 source .venv/bin/activate
4 pip install -r requirements.txt
5
6 export AZURE_SUBSCRIPTION_ID=...
7 export RESOURCE_GROUP=RG-PFE-SOC
8
9 # Cycle complet
10 make pipeline-run
11
12 # Sortie : build/dossier-AAAAAMJJ-HHMMSS.pdf
```

Listing 2 – Installation et premier cycle.

.6 Le modèle Evidence

```
1 class Evidence(BaseModel):
2     model_config = ConfigDict(frozen=True, extra="forbid")
3
4     id: UUID = Field(default_factory=uuid4)
5     ts: datetime = Field(
```

```
6         default_factory=lambda: datetime.now(timezone.utc)
7     )
8     source: SourceType
9     category: Category
10    ref: str = Field(..., min_length=1, max_length=255)
11    scope: Scope = Field(default_factory=Scope)
12    payload: dict[str, Any]
13    payload_hash: str = Field(..., pattern=r"^[a-f0-9]{64}$")
14    weight: Weight = Field(default=Weight.AUTO)
15    materiality: Materiality = Field(default=Materiality.MEDIUM)
16
17    @classmethod
18    def from_payload(cls, source, category, ref, payload, ...):
19        canonical = json.dumps(
20            payload, sort_keys=True, separators=(",", ":")
21        )
22        payload_hash = hashlib.sha256(
23            canonical.encode("utf-8")
24        ).hexdigest()
25        return cls(
26            source=source, category=category, ref=ref,
27            payload=payload, payload_hash=payload_hash, ...
28        )
```

Listing 3 – Modèle Pydantic Evidence (extrait).

7 Le diff

```
1 def diff(previous, current) -> list[Change]:
2     changes = []
3     prev_keys = set(previous.keys())
4     curr_keys = set(current.keys())
5
6     for key in curr_keys - prev_keys:
7         ev = current[key]
8         changes.append(
9             Change(type=ChangeType.CREATED,
10                  ref=ev.ref, category=ev.category,
11                  after=ev)
12         )
13     for key in prev_keys - curr_keys:
14         ev = previous[key]
15         changes.append(
16             Change(type=ChangeType.DELETED,
17                  ref=ev.ref, category=ev.category,
18                  before=ev)
19         )
20     for key in prev_keys & curr_keys:
21         if previous[key].payload_hash != current[key].payload_hash:
22             changes.append(
23                 Change(type=ChangeType.UPDATED,
```

```

24         ref=current[key].ref,
25         category=current[key].category,
26         before=previous[key], after=current[key])
27     )
28     return changes

```

Listing 4 – Moteur de diff (extrait).

.8 Workflow GitHub Actions (cycle nightly)

```

1  name: HOMO-CI nightly
2  on:
3    schedule:
4      - cron: '0 3 * * *'    # 03:00 UTC tous les jours
5    workflow_dispatch:
6
7  jobs:
8    run-pipeline:
9      runs-on: ubuntu-latest
10     steps:
11       - uses: actions/checkout@v4
12       - uses: actions/setup-python@v5
13       with:
14         python-version: '3.11'
15       - run: pip install -r pipeline/requirements.txt
16       - name: Run pipeline
17       env:
18         AZURE_SUBSCRIPTION_ID: ${ secrets.AZURE_SUB_ID }
19       run: |
20         cd pipeline
21         homo-ci collect
22         homo-ci diff
23         homo-ci render
24         homo-ci publish

```

Listing 5 – .github/workflows/nightly.yml (extrait).

.9 Dépendances principales

- pydantic >= 2.5 : validation des modèles Evidence, Change, DossierSnapshot.
- azure-identity, azure-mgmt-network, azure-mgmt-resource : SDK Azure officiel.
- jinja2 >= 3.1 : templating Markdown.
- py pandoc et weasyprint : conversion Markdown vers PDF.
- click et rich : CLI et affichage.

Données et catalogue de référence

.10 Catalogue des 100 preuves attendues

Le catalogue de référence pour la mesure de couverture a été construit par lecture du *Guide d'homologation ANSSI* et de l'*ISO/IEC 27002 :2022*. Chaque preuve est classée :

- **A** : automatisable (collectable par un script depuis une API ou un fichier).
- **B** : semi-automatisable (collectable mais avec intervention humaine pour valider la pertinence).
- **C** : manuelle (formation, sensibilisation, revue d'analyse de risque, signature d'un document).

.11 Décomposition par étape ANSSI

Table 1 – Répartition des 100 preuves par étape ANSSI.

Étape ANSSI	Total	A	B	C	Couvert
1. Cadrage	6	0	1	5	0
2. Périmètre fonctionnel	5	0	2	3	1
3. Acteurs et rôles	7	1	2	4	1
4. Cartographie SI	18	14	3	1	14
5. Analyse de risque EBIOS RM	8	0	3	5	1
6. Mesures de sécurité	28	19	6	3	16
7. Audits et tests	16	11	4	1	9
8. Décision d'homologation	0	0	0	0	0
9. Maintien dans le temps	12	7	3	2	5
Total	100	52	24	24	47

Note : l'étape 8 (décision d'homologation) est par nature un acte signé par l'AQSSI, non automatisable. Elle ne compte pas dans le score.

.12 Exemple de preuves par catégorie

.12.1 Catégorie A (automatisable)

- Liste des règles NSG et leur état **Allow/Deny**.
- Liste des CVE **HIGH** et **CRITICAL** sur les images Docker en production.
- Liste des comptes AD inactifs (**lastLogon** > 90 jours).
- Liste des règles Wazuh activées par catégorie.
- État des agents Wazuh (nombre, dernière connexion).

.12.2 Catégorie B (semi-automatisable)

- Cartographie des flux entre applications (peut être extraite des logs nginx, mais demande validation manuelle).
- Inventaire des comptes à privilèges (extractible LDAP, mais la qualification « *privilegié* » demande revue).
- Liste des journaux activés sur les composants critiques (à corréler avec le catalogue attendu).

.12.3 Catégorie C (manuelle)

- Compte-rendu des sessions de sensibilisation des utilisateurs.
- Procédure de gestion des incidents et son test annuel.
- Compte-rendu de l'atelier EBIOS RM atelier 3 (scénarios stratégiques).
- Signature de l'AQSSI sur la décision d'homologation.

.13 Données de simulation pour l'audit drift

La simulation rétroactive de 180 jours utilise un modèle de changement calibré sur le journal git du lab :

Table 2 – Taux de changement mensuel par catégorie (calibré sur le lab).

Catégorie	Changements / mois
Règles NSG	2–4
CVE (apparition + correction)	8–15
Comptes AD	1–3
Règles Wazuh	1–2

Ces taux sont volontairement bas (lab modeste). Sur un ENT en production, ils seraient probablement multipliés par 5 à 10, ce qui rendrait l'écart entre dossier statique et dossier continu encore plus marqué.